

The `compatible` property

- Is a list of strings
  - From the most specific to the least specific
- Describes the specific **binding** to which the node complies.
- It uniquely identifies the **programming model** of the device.
- Practically speaking, it is used by the operating system to find the **appropriate driver** for this device.
- When describing real hardware, the typical form is `vendor,model`
- Examples:
  - `compatible = "arm,armv7-timer";`
  - `compatible = "st,stm32mp1-dwmac", "snps,dwmac-4.20a";`
  - `compatible = "regulator-fixed";`
  - `compatible = "gpio-keys";`
- Special value: `simple-bus` → bus where all sub-nodes are memory-mapped devices



`compatible` property and Linux kernel drivers

- Linux identifies as **platform devices**:
  - Top-level DT nodes with a `compatible` string
  - Sub-nodes of `simple-bus`
    - Instantiated automatically at boot time
- Sub-nodes of I2C controllers → *I2C devices*
- Sub-nodes of SPI controllers → *SPI devices*
- Each Linux driver has a table of compatible strings it supports
  - `struct of_device_id[]`
- When a DT node compatible string matches a given driver, the device is *bound* to that driver.

```
/ {
    timer {                                  → Platform device
        compatible = "...";
    };
    soc {
        compatible = "simple-bus";
        uart@1000 {                          → Platform device
            compatible = "...";
        };
        i2c@2000 {                           → Platform device
            compatible = "...";
            eeprom@65 {                      → I2C device
                compatible = "...";
            };
        };
    };
};
```



Matching with drivers in Linux: platform driver

drivers/tty/serial/stm32-usart.c

```
static const struct of_device_id stm32_match[] = {
        { .compatible = "st,stm32-uart", .data = &stm32f4_info},
        { .compatible = "st,stm32f7-uart", .data = &stm32f7_info},
        { .compatible = "st,stm32h7-uart", .data = &stm32h7_info},
        {},
}; MODULE_DEVICE_TABLE(of, stm32_match);

...

static struct platform_driver stm32_serial_driver = {
        .probe          = stm32_serial_probe,
        .remove         = stm32_serial_remove,
        .driver = {
                .name   = DRIVER_NAME,
                .pm     = &stm32_serial_pm_ops,
                .of_match_table = of_match_ptr(stm32_match),
        },
};
```



Matching with drivers in Linux: I2C driver

sound/soc/codecs/cs42l51.c

```
const struct of_device_id cs42l51_of_match[] = {
        { .compatible = "cirrus,cs42l51", },
        { }
}; MODULE_DEVICE_TABLE(of, cs42l51_of_match);
```

sound/soc/codecs/cs42l51-i2c.c

```
static struct i2c_driver cs42l51_i2c_driver = {
        .driver = {
                .name = "cs42l51",
                .of_match_table = cs42l51_of_match,
                .pm = &cs42l51_pm_ops,
        },
        .probe = cs42l51_i2c_probe,
        .remove = cs42l51_i2c_remove,
        .id_table = cs42l51_i2c_id,
};
```



reg property

- Most important property after `compatible`
- **Memory-mapped** devices: base physical address and size of the memory-mapped registers. Can have several entries for multiple register areas.

```
sai4: sai@50027000 {
    reg = <0x50027000 0x4>, <0x500273f0 0x10>;
};
```



reg property

- Most important property after `compatible`
- **Memory-mapped** devices: base physical address and size of the memory-mapped registers. Can have several entries for multiple register areas.
- **I2C** devices: address of the device on the I2C bus.

```
&i2c1 {
    hdmi-transmitter@39 {
        reg = <0x39>;
    };
    cs42l51: cs42l51@4a {
        reg = <0x4a>;
    };
};
```



reg property

- Most important property after `compatible`
- **Memory-mapped** devices: base physical address and size of the memory-mapped registers. Can have several entries for multiple register areas.
- **I2C** devices: address of the device on the I2C bus.
- **SPI** devices: chip select number

```
&qspi {
        flash0: mx66l51235l@0 {
                reg = <0>;
        };
        flash1: mx66l51235l@1 {
                reg = <1>;
        };
};
```



reg property

- Most important property after `compatible`
- **Memory-mapped** devices: base physical address and size of the memory-mapped registers. Can have several entries for multiple register areas.
- **I2C** devices: address of the device on the I2C bus.
- **SPI** devices: chip select number
- The unit address must be the address of the first `reg` entry.

```
sai4: sai@50027000 {
    reg = <0x50027000 0x4>, <0x500273f0 0x10>;
};
```



Status property

- The `status` property indicates if the device is really in use or not
  - `okay` or `ok` → the device is really in use
  - any other value, by convention `disabled` → the device is not in use
- In Linux, controls if a device is instantiated
- In `.dtsi` files describing SoCs: all devices that interface to the outside world have: `status = "disabled";`
- Enabled on a per-device basis in the board `.dts`



Resources: interrupts, clocks, DMA, reset lines, ...

- Common pattern for resources shared by multiple hardware blocks
  - Interrupt lines
  - Clock controllers
  - DMA controllers
  - Reset controllers
  - ...
- A Device Tree node describing the *controller* as a device
- References from other nodes that use resources provided by this *controller*

```
intc: interrupt-controller@a0021000 {
   compatible = "arm,cortex-a7-gic";
   #interrupt-cells = <3>;
   interrupt-controller;
   reg = <0xa0021000 0x1000>, <0xa0022000 0x2000>;
};

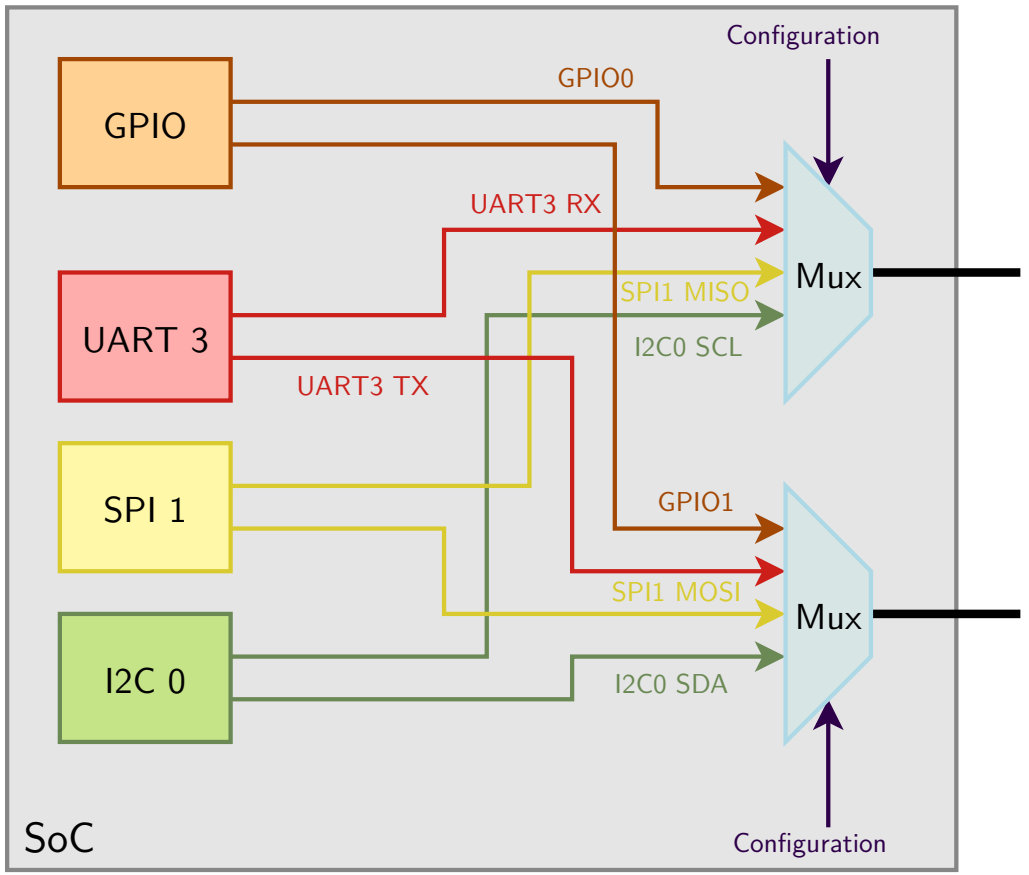
rcc: rcc@50000000 {
   compatible = "st,stm32mp1-rcc", "syscon";
   reg = <0x50000000 0x1000>;
   #clock-cells = <1>;
   #reset-cells = <1>;
};

dmamux1: dma-router@48002000 {
   compatible = "st,stm32h7-dmamux";
   reg = <0x48002000 0x1c>;
   #dma-cells = <3>;
   clocks = <&rcc DMAMUX>;
   resets = <&rcc DMAMUX_R>;
};

spi3: spi@4000c000 {
   interrupts = <GIC_SPI 51 IRQ_TYPE_LEVEL_HIGH>;
   clocks = <&rcc SPI3_K>;
   resets = <&rcc SPI3_R>;
   dmas = <&dmamux1 61 0x400 0x05>,  <&dmamux1 62 0x400 0x05>;
};
```

Pin-muxing description

- Most modern SoCs, including the STM32MP1, have more features than they have pins to expose those features to the outside world.
- Pins are muxed: a given pin can be used for one function **or** another
- A specific IP block in the SoC controls the muxing of pins: the **pinmux controller**
- The Device Tree describes which pin configurations are possible, and which configurations are used by the different devices.





Pin-muxing controllers on STM32MP1

arch/arm/boot/dts/st/stm32mp151.dtsi

```
pinctrl: pin-controller@50002000 {
        compatible = "st,stm32mp157-pinctrl";
        ...
};

pinctrl_z: pin-controller-z@54004000 {
        compatible = "st,stm32mp157-z-pinctrl";
        ...
};
```



Pin-muxing configuration

arch/arm/boot/dts/st/stm32mp15-pinctrl.dtsi

```
&pinctrl {
        ...
        i2c1_pins_a: i2c1-0 {
                pins {
                        pinmux = <STM32_PINMUX('D', 12, AF5)>, /* I2C1_SCL */
                                 <STM32_PINMUX('F', 15, AF5)>; /* I2C1_SDA */
                        bias-disable;
                        drive-open-drain;
                        slew-rate = <0>;
                };
        };
        ...
        m_can1_pins_a: m-can1-0 {
                pins1 {
                        pinmux = <STM32_PINMUX('H', 13, AF9)>; /* CAN1_TX */
                        slew-rate = <1>;
                        drive-push-pull;
                        bias-disable;
                };
                pins2 {
                        pinmux = <STM32_PINMUX('I', 9, AF9)>; /* CAN1_RX */
                        bias-disable;
                };
        };
        ...
};
```



Pin-muxing configuration

Table 8. Alternate function AF0 to AF7[1] (continued)

Port		AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7
		HDP/SYS/RTC	TIM1/2/16/17/ LPTIM1/SYS/ RTC	SAI1/4/I2C6/ TIM3/4/5/12/ HDP/SYS	SAI4/I2C2/ TIM8/ LPTIM2/3/4/5/ DFSDM1 /SDMMC1	SAI4/ I2C1/2/3/4/5/ USART1/ TIM15/LPTIM2/ DFSDM1/CEC	SPI1/I2S1/ SPI2/I2S2/ SPI3/I2S3/ SPI4/5/6/I2C1/ SDMMC1/3/ CEC	SPI3/I2S3/ SAI1/3/4/ I2C4/UART4/ DFSDM1	SPI2/I2S2/ SPI3/I2S3/ SPI6/ USART1/2/3/6/ UART7/ SDMMC2
Port D	PD6	-	TIM16_CH1N	SAI1_D1	DFSDM1_ CKIN4	DFSDM1_ DATIN1	SPI3_MOSI/ I2S3_SDO	SAI1_SD_A	USART2_RX
	PD7	TRACED6	-	-	DFSDM1_ DATIN4	I2C2_SCL	-	DFSDM1_ CKIN1	USART2_CK
	PD8	-	-	-	DFSDM1_ CKIN3	-	-	SAI3_SCK_B	USART3_TX
	PD9	-	-	-	DFSDM1_ DATIN3	-	-	SAI3_SD_B	USART3_RX
	PD10	RTC_REFIN	TIM16_BKIN	-	DFSDM1_ CKOUT	I2C5_SMBA	SPI3_MISO/ I2S3_SDI	SAI3_FS_B	USART3_CK
	PD11	-	-	-	LPTIM2_IN2	I2C4_SMBA	I2C1_SMBA	-	USART3_CTS/ USART3_NSS
	PD12	-	LPTIM1_IN1	TIM4_CH1	LPTIM2_IN1	I2C4_SCL	I2C1_SCL	-	USART3_RTS/ USART3_DE
	PD13	-	LPTIM1_OUT	TIM4_CH2	-	I2C4_SDA	I2C1_SDA	I2S3_MCK	-
	PD14	-	-	TIM4_CH3	-	-	-	SAI3_MCLK_B	-
	PD15	-	-	TIM4_CH4	-	-	-	SAI3_MCLK_A	-
Port E	PE0	-	LPTIM1_ETR	TIM4_ETR	-	LPTIM2_ETR	SPI3_SCK/ I2S3_CK	SAI4_MCLK_B	-
	PE1	-	LPTIM1_IN2	-	-	-	I2S2_MCK	SAI3_SD_B	-
	PE2	TRACECLK	-	SAI1_CK1	-	I2C4_SCL	SPI4_SCK	SAI1_MCLK_A	-
	PE3	TRACED0	-	-	-	TIM15_BKIN	-	SAI1_SD_B	-

Source: STM32MP157C datasheet. Note that `I2C1_SDA` is also available on pin `PF15` (not shown here).



Pin-muxing consumer

```
&i2c1 {
        pinctrl-names = "default", "sleep";
        pinctrl-0 = <&i2c1_pins_a>;
        pinctrl-1 = <&i2c1_sleep_pins_a>;
        ...
};
```

- Typically board-specific, in `.dts`
- `pinctrl-0`, `pinctrl-1`, `pinctrl-X` provides the pin mux configurations for the different **states**
- `pinctrl-names` gives a name to each state, mandatory even if only one state
- States are mutually exclusive
- The driver is responsible for switching between states
- `default` state is automatically set up when the device is *probed*



Example: LED and I2C device

- Let's see how to describe an LED and an I2C device connected to the DK1 platform.
- Create `arch/arm/boot/dts/st/stm32mp157a-dk1-custom.dts` which includes `stm32mp157a-dk1.dts`

```
#include "stm32mp157a-dk1.dts"
```

- Make sure `stm32mp157a-dk1-custom.dts` gets compiled to a DTB by changing arch/arm/boot/dts/Makefile

```
dtb-$(CONFIG_ARCH_STM32) +=
        ...
        stm32mp157a-dk1.dtb
        stm32mp157a-dk1-custom.dtb
```

- `make dtbs`

```
  DTC     arch/arm/boot/dts/st/stm32mp157a-dk1-custom.dtb
```



Example: describe an LED

stm32mp157a-dk1-custom.dts

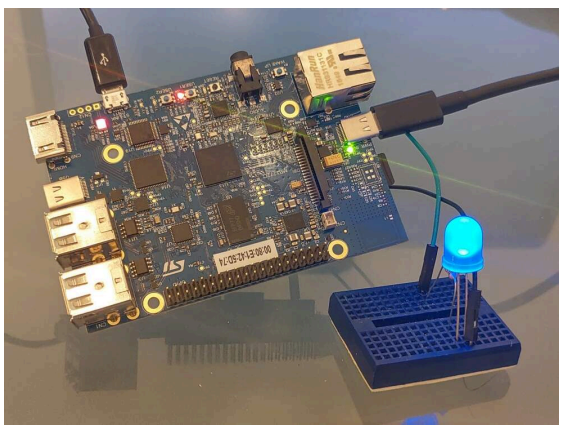
```
#include "stm32mp157a-dk1.dts"

/ {
        leds {
                compatible = "gpio-leds";
                webinar {
                        label = "webinar";
                        gpios = <&gpioe 1 GPIO_ACTIVE_HIGH>;
                };
        };
};
```

shell

```
# echo 255 > /sys/class/leds/webinar/brightness
```

1	ARD_D0	PE7	USART7_RX
2	ARD_D1	PE8	USART7_TX
3	ARD_D2	PE1	IO
4	ARD_D3	PD14	TIM4_CH3
5	ARD_D4	PE10	IO
6	ARD_D5	PD15	TIM4_CH4
7	ARD_D6	PE9	TIM1_CH1
8	ARD_D7	PD1	IO



Example: connect I2C temperature, humidity and pressure sensor

stm32mp157a-dk1-custom.dts

```
&i2c5 {
        status = "okay";
        clock-frequency = <100000>;
        pinctrl-names = "default", "sleep";
        pinctrl-0 = <&i2c5_pins_a>;
        pinctrl-1 = <&i2c5_pins_sleep_a>;

        pressure@76 {
                compatible = "bosch,bme280";
                reg = <0x76>;
        };
};
```

shell

```
# cat /sys/bus/iio/devices/iio:device2/
in_humidityrelative_input
49147
# cat /sys/bus/iio/devices/iio:device2/in_pressure_input
101.567167968
# cat /sys/bus/iio/devices/iio:device2/in_temp_input
24380
```

CN13	1	ARD_D8	PG3	IO
	2	ARD_D9	PH6	TIM12_CH1
	3	ARD_D10	PE11	SPI4_NSS and TIM1_CH2
	4	ARD_D11	PE14	SPI4_MOSI and TIM1_CH4
	5	ARD_D12	PE13	SPI4_MISO
	6	ARD_D13	PE12	SPI4_SCK
	7	GND	-	GND
	8	VREFP	-	VREF+
	9	ARD_D14	PA12	I2C5_SDA
	10	ARD_D15	PA11	I2C5_SCL

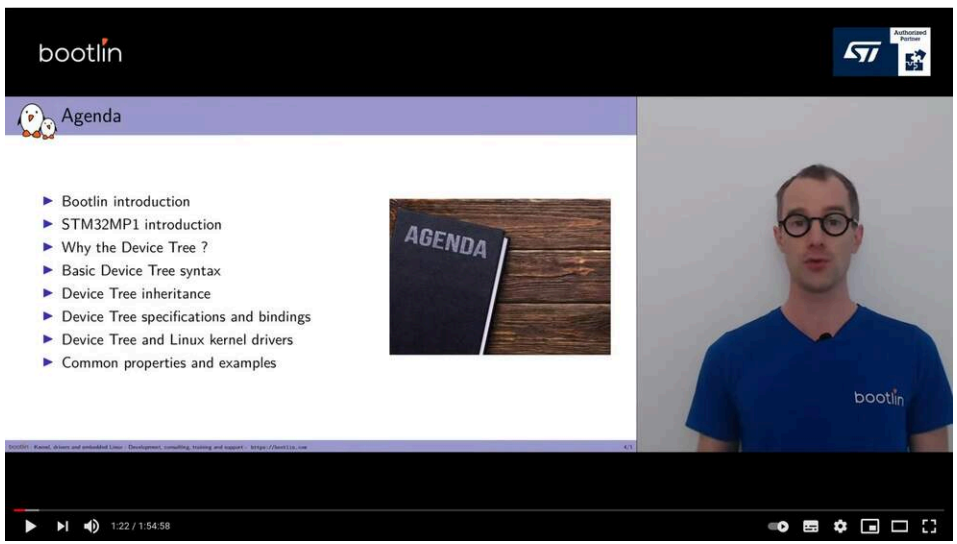


Details at https://bootlin.com/blog/building-a-linux-system-for-the-stm32mp1-connecting-an-i2c-sensor/

Further details about the Device Tree

Check out our *DeviceTree 101 webinar*, by Thomas Petazzoni (2021)

- Slides: https://bootlin.com/blog/device-tree-101-webinar-slides-and-videos/
- Video: https://youtu.be/a9CZ1Uk3OYQ





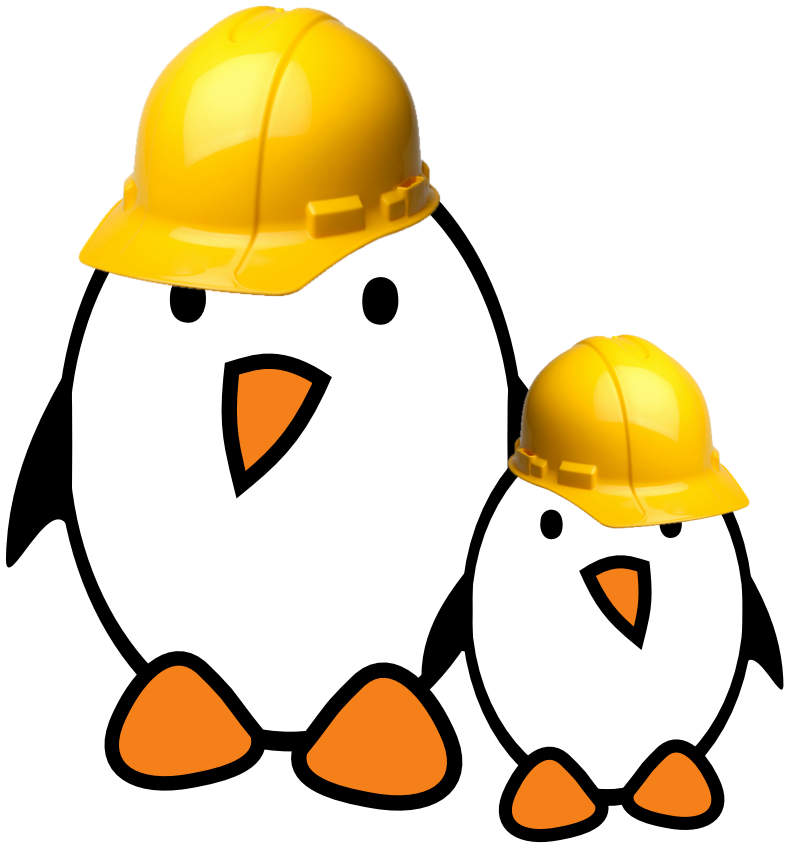
Discoverable hardware: USB and PCI



Discoverable hardware

- Some busses have built-in hardware discoverability mechanisms
- Most common busses: USB and PCI
- Hardware devices can be enumerated, and their characteristics retrieved with just a driver or the bus controller
- Useful Linux commands
  - `lsusb`, lists all USB devices detected
  - `lspci`, lists all PCI devices detected
  - A detected device does not mean it has a kernel driver associated to it!
- Association with kernel drivers done based on product ID/vendor ID, or some other characteristics of the device: device class, device sub-class, etc.

Practical lab - Accessing hardware devices



Time to start the practical lab!

- Exploring the contents of `/dev` and `/sys` and the devices available on the embedded hardware platform.
- Using GPIOs and LEDs.
- Modifying the Device Tree to control pin multiplexing and declare an I2C-connected joystick.
- Adding support for a USB audio card using Linux kernel modules
- Adding support for the I2C-connected joystick through an out-of-tree module.